

Technische Dokumentation — Lernzeit-Manager

Projekt: Lernzeit-Manager · ISEF01

Meilenstein: MS 4

Team: Elias Ebertshäuser · Assis Ramadan · Julian Wagner

Datum: August 2026

1. Systemarchitektur

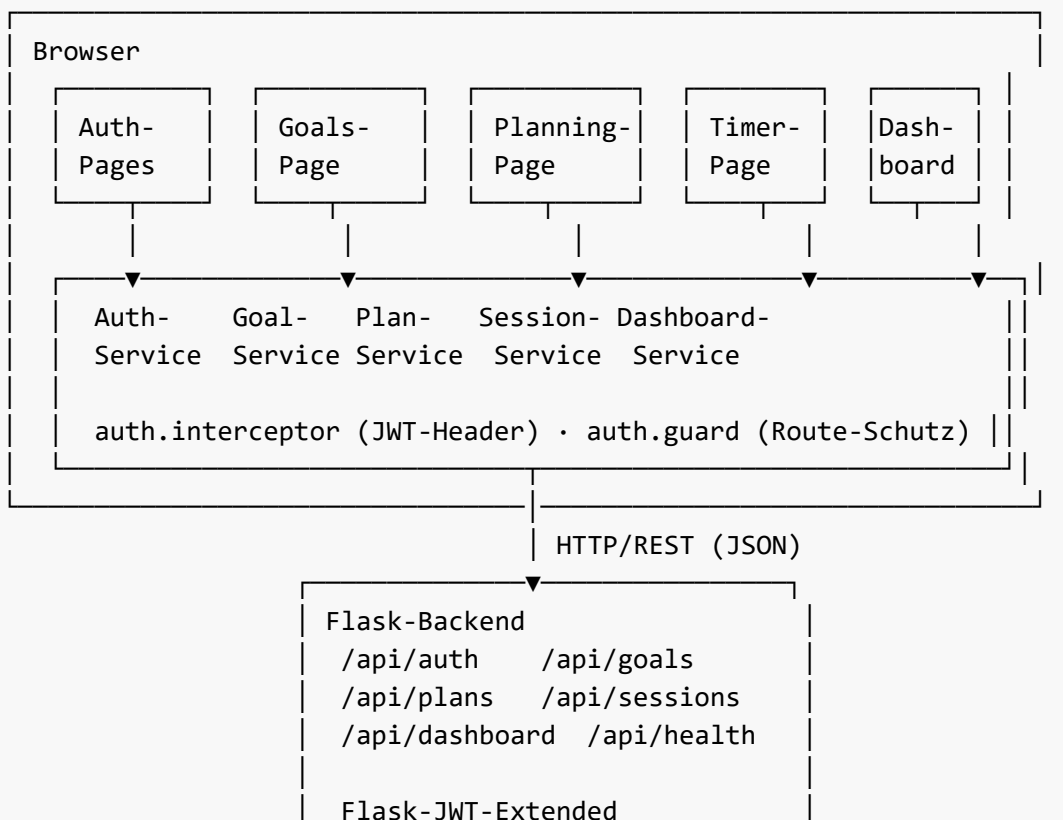
1.1 Übersicht

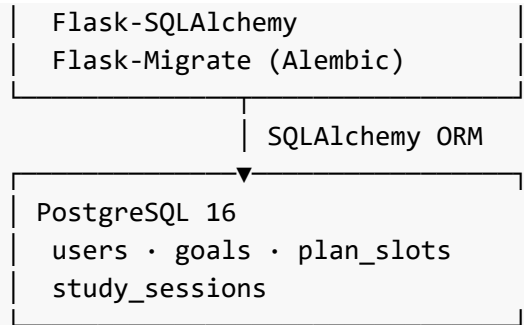
Der Lernzeit-Manager folgt einer klassischen **dreischichtigen Client-Server-Architektur**:

```
[Browser] ↔ [Flask-Backend (REST-API)] ↔ [PostgreSQL-Datenbank]
Angular SPA   Python 3.12, Port 5000/8000   Relationale Datenbank
```

- **Frontend (Angular 22 SPA):** Eine Single-Page-Application, die vollständig im Browser läuft. Sie kommuniziert ausschließlich über HTTP-REST mit dem Backend.
- **Backend (Flask 3.1 REST-API):** Eine Python-Anwendung, die Authentifizierung, Datenlogik und Datenbankzugriff übernimmt. Sie gibt JSON zurück.
- **Datenbank (PostgreSQL 16):** Speichert alle Nutzer-, Ziel-, Planungs- und Session-Daten dauerhaft.

1.2 Komponentendiagramm (Textbeschreibung)





1.3 Kommunikation

Das Frontend sendet alle API-Anfragen an relative URLs (z. B. `/api/goals`). Im Entwicklungsmodus leitet ein Angular-Dev-Proxy diese an `http://localhost:5000` weiter. In Produktion liefert Flask sowohl die API als auch die statischen Angular-Dateien aus demselben Server aus — es gibt keine Cross-Origin-Problematik.

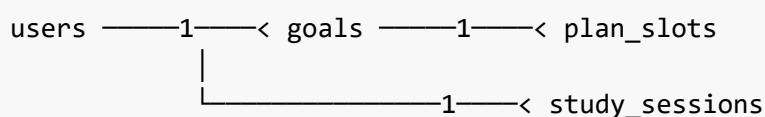
Alle Anfragen an geschützte Endpunkte enthalten einen JWT-Token im HTTP-Header:

```
Authorization: Bearer <token>
```

Der Token wird beim Login ausgestellt, im `localStorage` des Browsers gespeichert (`lm_token`) und von einem Angular-Interceptor automatisch an jede ausgehende Anfrage angehängt.

2. Datenbankschema

2.1 Entity-Relationship-Übersicht



Jeder Nutzer besitzt beliebig viele Ziele. Jedes Ziel besitzt beliebig viele Planungseinträge und Lernsessions. Beim Löschen eines Nutzers oder Ziels werden zugehörige Datensätze automatisch mitgelöscht (Cascade Delete).

2.2 Tabellenbeschreibung

users

Spalte	Typ	Beschreibung
id	INTEGER PK	Primärschlüssel
email	VARCHAR(255) UNIQUE	E-Mail-Adresse (Login)
name	VARCHAR(255)	Anzeigename

Spalte	Typ	Beschreibung
password_hash	VARCHAR(255)	bcrypt-Hash des Passworts

goals

Spalte	Typ	Beschreibung
id	INTEGER PK	Primärschlüssel
user_id	INTEGER FK→users	Besitzer
title	VARCHAR(255)	Titel des Lernziels
module_name	VARCHAR(255)	Modulkürzel oder -name
ects	INTEGER	ECTS-Punkte (1–30)
status	VARCHAR(50)	open / in_progress / achieved
target_date	DATE	Angestrebter Abschlusstermin
created_at	DATETIME	Anlage-Zeitpunkt

plan_slots

Spalte	Typ	Beschreibung
id	INTEGER PK	Primärschlüssel
user_id	INTEGER FK→users	Besitzer
goal_id	INTEGER FK→goals	Zugehöriges Lernziel
year	INTEGER	Jahr der Planung
month	INTEGER	Monat der Planung (1–12)
day	INTEGER NULL	Optionaler Tag (1–31)
planned_time	VARCHAR(5) NULL	Optionale Uhrzeit (HH:MM)
duration_minutes	INTEGER	Geplante Lernzeit in Minuten
note	TEXT NULL	Optionale Notiz

study_sessions

Spalte	Typ	Beschreibung
id	INTEGER PK	Primärschlüssel
user_id	INTEGER FK→users	Besitzer
goal_id	INTEGER FK→goals	Zugehöriges Lernziel
started_at	DATETIME	UTC-Startzeit (naive)

Spalte	Typ	Beschreibung
paused_at	DATETIME NULL	Zeitpunkt der aktuellen Pause
total_paused_seconds	INTEGER	Summierte Pausendauer aller Pausen
ended_at	DATETIME NULL	UTC-Endzeit
duration_seconds	INTEGER NULL	Berechnete Netto-Lernzeit
status	VARCHAR(20)	active / paused / completed
note	TEXT NULL	Optionale Notiz

3. API-Beschreibung

Alle Endpunkte liegen unter dem Präfix `/api`. Geschützte Endpunkte erfordern den `Authorization: Bearer <token>`-Header.

3.1 Authentifizierung

POST /api/auth/register

Neuen Nutzer registrieren. Kein Token erforderlich.

Request:

```
{
  "email": "nutzer@example.com",
  "name": "Max Mustermann",
  "password": "Sicher123"
}
```

Response (201):

```
{ "access_token": "...", "user": { "id": 1, "email": "...", "name": "..." } }
```

Fehler: 400 (Fehlende Felder, Passwort < 6 Zeichen), 409 (E-Mail bereits registriert)

POST /api/auth/login

Mit Zugangsdaten einloggen. Kein Token erforderlich.

Request: { "email": "...", "password": "..." }

Response (200): { "access_token": "...", "user": { ... } }

Fehler: 400, 401 (Falsches Passwort)

GET /api/auth/me

Daten des eingeloggten Nutzers abrufen. Token erforderlich.

Response (200): { "id": 1, "email": "...", "name": "..." }

3.2 Lernziele

GET /api/goals — Alle Ziele des Nutzers abrufen (Token erforderlich)

Response (200): [{ "id": 1, "title": "...", "module_name": "...", "ects": 5, "status": "open", "target_date": "2026-12-31", "created_at": "..." }, ...]

POST /api/goals — Neues Ziel anlegen

Request: { "title": "...", "module_name": "...", "ects": 5, "status": "open", "target_date": "2026-12-31" }

Response (201): Das erstellte Goal-Objekt

Fehler: 400 (Pflichtfelder fehlen)

PUT /api/goals/ — Ziel aktualisieren (Status, Titel, usw.)

Request: { "status": "in_progress" } (nur zu ändernde Felder)

Response (200): Das aktualisierte Goal-Objekt

Fehler: 400, 404

DELETE /api/goals/ — Ziel löschen (inkl. Sessions und Planungseinträge)

Response (204)

Fehler: 404

3.3 Planung

GET /api/plans — Planungseinträge abrufen

Query-Parameter: `goal_id` (optional), `year` (optional), `month` (optional)

Response (200): [{ "id": 1, "goal_id": 1, "year": 2026, "month": 8, "day": 15, "planned_time": "18:00", "duration_minutes": 90, "note": null }, ...]

POST /api/plans — Neuen Planungseintrag anlegen

Request: { "goal_id": 1, "year": 2026, "month": 8, "duration_minutes": 60 }

Response (201): Der erstellte PlanSlot

Fehler: 400, 404 (Goal nicht gefunden)

DELETE /api/plans/ — Planungseintrag löschen

Response (204)

Fehler: 404

3.4 Lernsessions

GET /api/sessions/active — Aktive oder pausierte Session abrufen

Response (200): Session-Objekt mit `goal_title`

Response (204): Keine aktive Session

GET /api/sessions — Abgeschlossene Sessions abrufen

Query-Parameter: `goal_id` (optional), `limit` (optional, Standard 50)

Response (200): Liste von Session-Objekten

POST /api/sessions/start — Neue Session starten

Request: { `"goal_id": 1` }

Response (201): Die gestartete Session

Fehler: 400, 404, 409 (Bereits eine Session aktiv)

POST /api/sessions//pause — Session pausieren

Response (200): Aktualisierte Session

Fehler: 409 (Session nicht aktiv)

POST /api/sessions//resume — Pausierte Session fortsetzen

Response (200): Aktualisierte Session (mit aktualisierten `total_paused_seconds`)

Fehler: 409 (Session nicht pausiert)

POST /api/sessions//stop — Session beenden und Dauer berechnen

Request (optional): { `"note": "..."` }

Response (200): Abgeschlossene Session mit `duration_seconds`

Fehler: 409 (Session bereits beendet)

3.5 Dashboard

GET /api/dashboard — Aggregierte Monatsdaten abrufen

Response (200):

```
{
  "current_month": {
    "year": 2026, "month": 8,
    "planned_minutes": 300,
    "actual_minutes": 120
  },
  "goals": [
```

```
{
  "id": 1, "title": "Mathematik", ...,
  "total_actual_minutes": 120,
  "planned_ects_minutes": 9000
},
"inactivity_warning": false,
"active_session": null
}
```

3.6 Health Check

GET /api/health — Statusprüfung des Backends

Response (200): { "status": "ok" }

4. Frontend-Architektur

4.1 Verzeichnisstruktur

```
frontend/src/app/
├── core/
│   ├── models/index.ts      TypeScript-Interfaces (User, Goal, PlanSlot, ...)
│   ├── services/            HTTP-Services (AuthService, GoalService, ...)
│   ├── interceptors/        auth.interceptor.ts (JWT-Header-Injektion)
│   └── guards/              auth.guard.ts (Routen-Schutz)
├── features/
│   ├── auth/                Login- und Registrierungskomponenten
│   ├── dashboard/           Dashboard-Komponente
│   ├── goals/               Lernziel-Verwaltung
│   ├── planning/            Planungs-Verwaltung
│   └── timer/               Timer-Komponente
└── layout/
    └── navbar/              Navigationsleiste
```

4.2 Wichtige Konzepte

Standalone-Komponenten: Angular 22 verwendet keine NgModule mehr. Jede Komponente deklariert ihre Abhängigkeiten (`imports: [RouterLink, FormsModule, ...]`) direkt.

Signals: Reaktive Zustände werden über `signal<T>()` verwaltet. Ändert sich ein Signal, aktualisiert Angular nur die betroffene Stelle im DOM.

Lazy Loading: Alle Feature-Routen werden erst bei Bedarf geladen, was die initiale Ladezeit der App verkürzt.

JWT-Interceptor: Ein funktionaler Angular-Interceptor (`HttpInterceptorFn`) hängt automatisch den JWT-Token an jede ausgehende HTTP-Anfrage an.

Auth-Guard: Eine `CanActivateFn`-Funktion prüft bei jedem Seitenwechsel, ob ein Token im `localStorage` vorhanden ist. Fehlt er, wird auf `/login` weitergeleitet.

5. Deployment-Infrastruktur

5.1 Lokale Entwicklungsumgebung

```
Docker Compose → PostgreSQL 16 (Port 5432)
Python 3.12 + virtualenv → Flask Dev-Server (Port 5000)
Node.js 22 + npm → Angular Dev-Server (Port 4200, Proxy auf 5000)
```

5.2 Produktionsumgebung (Railway)

Railway ist eine Plattform-as-a-Service, die Docker-Container in der Cloud betreibt.

Build-Prozess (Nixpacks):

1. Nixpacks liest `nixpacks.toml` aus dem Repository-Root
2. Python 3 und Hilfspakete werden über `apt` installiert; Node.js 22 wird über das NodeSource-Setup-Skript (`curl | bash + apt-get install nodejs`) bereitgestellt
3. Ein Python-Virtualenv (`/app/.venv`) wird angelegt und `pip install -r backend/requirements.txt` ausgeführt
4. `npm --prefix frontend ci` installiert Angular-Abhängigkeiten
5. `npm --prefix frontend run build` erstellt den Angular-Produktions-Build (Output: `frontend/dist/frontend/browser/`)

Start-Prozess (`start.sh`):

1. `flask db upgrade` spielt ausstehende Datenbankmigrationen ein (idempotent)
2. `gunicorn wsgi:application -w 2 -b 0.0.0.0:$PORT` startet den Produktionsserver

Statische Dateien: In Produktion liefert Flask die Angular-Dateien aus `frontend/dist/frontend/browser/` aus. Alle Anfragen, die nicht mit `/api/` beginnen, werden auf `index.html` weitergeleitet (SPA-Routing).

Umgebungsvariablen: Alle Secrets (Datenbankpasswort, JWT-Secret) werden als Railway Environment Variables konfiguriert — nie im Repository gespeichert.

5.3 CI/CD-Pipeline (GitHub Actions)

Bei jedem Push auf `main` oder Feature-Banches:

1. Backend-Job: `ruff check` (Linting) + `pytest` (13 Tests)
2. Frontend-Job: `ng lint` (ESLint) + `ng test` (Vitest, 18 Tests)

Bei grüner CI und Merge auf `main`: automatischer Deploy auf Railway.